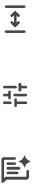




Bitácora Técnica y Mapa de Ruta Backend



Proyecto: LegalDocs Pro (Sistema LegalTech de Gestión de Contratos) **Stack:** .NET 10, C#, SQL Server, Entity Framework Core 8/10 **Arquitectura:** Clean Architecture + CQRS + Patrón Repositorio



1. Arquitectura del Proyecto (Implementada)

Hemos estructurado la solución basándonos en **Clean Architecture**, asegurando que la lógica de negocio esté completamente aislada de los detalles de infraestructura.

- `LegalDocsPro.Domain` (**Capa Central**): C# puro. Contiene entidades de dominio rico, enums y las interfaces (contratos) de los repositorios. No tiene dependencias de paquetes externos.
- `LegalDocsPro.Application` (**Casos de Uso**): Implementa **CQRS** utilizando `MediatR`. Aquí residen los *Commands* (escrituras) y *Queries* (lecturas). Conoce al Dominio, pero no sabe cómo se guardan los datos.
- `LegalDocsPro.Infrastructure` (**Detalles Técnicos**): Implementa el acceso a datos con **Entity Framework Core**. Contiene el `ApplicationDbContext`, las migraciones y la implementación de los repositorios.
- `LegalDocsPro.Api` (**Presentación**): Proyecto de ASP.NET Core Web API. Expone los endpoints, configura la Inyección de Dependencias (DI) en `Program.cs` y documenta con Swagger.



2. Hitos Completados (Hasta la fecha)

A. Diseño Inicial y Base de Datos

- [x] Diseño conceptual del modelo relacional (Contratos, Usuarios, Roles, Auditoría).
- [x] Decisión estratégica: Uso de **System-Versioned Temporal Tables** de SQL Server para auditoría inalterable.
- [x] Adopción del enfoque **Code-First** para que C# sea la única fuente de verdad, gestionando la DB mediante Migraciones de EF Core.

B. Capa de Dominio (Rich Domain Model)

- [x] Creación de `BaseEntity.cs` para centralizar campos de auditoría (`Id`, `CreatedAt`, etc.).
- [x] Resolución de problemas de accesibilidad (`internal` vs `public class`).
- [x] Creación de la entidad principal `Contract.cs` con **encapsulación estricta** (constructores privados, `private set`) y reglas de negocio nativas (ej. `Approve()`, `SendToReview()`).
- [x] Creación del enum `ContractStatus` (`Draft`, `InReview`, `Approved`, etc.).
- [x] Definición de `ICollectionRepository.cs`.

C. Capa de Infraestructura

- [x] Configuración de `ApplicationDbContext.cs`.
- [x] Mapeo de la tabla `Contracts` activando explícitamente `.IsTemporal()` para el historial automático.
- [x] Intercepción de `SaveChangesAsync` para auto-llenar fechas de auditoría (`CreatedAt`, `LastModifiedAt`).
- [x] Implementación de `ContractRepository.cs`.
- [x] Generación y ejecución exitosa de la migración `InitialCreate` hacia la base de datos local.

D. Capa de Aplicación y API (CQRS)

- [x] Creación del `CreateContractCommand` (DTO de transporte inmutable usando `record`).
- [x] Implementación del `CreateContractCommandHandler` para coordinar el guardado sin ensuciar el controlador.
- [x] Configuración de la inyección de dependencias en `Program.cs` (`AddDbContext`, `AddScoped`, `AddMediatR`).
- [x] Creación de `ContractsController.cs` con el primer endpoint: `POST /api/Contracts`.
- [x] Prueba exitosa de Inserción (Código 201) desde Swagger verificando el impacto en SQL Server.



3. Mapa de Ruta (Lo que falta por construir)

A continuación, el orden lógico sugerido para completar el MVP (Producto Mínimo Viable) del backend:

Fase 1: Completar CRUD de Contratos y Validaciones (Inmediato)

- [] **Validaciones (FluentValidation):** Crear `CreateContractCommandValidator` para asegurar que no se creen contratos vacíos o con fechas inválidas antes de que lleguen al dominio.
- [] **Queries (Lecturas):**
 - Endpoint `GET /api/contracts/{id}` (Consultar por ID).
 - Endpoint `GET /api/contracts` (Listar contratos con paginación básica).
- [] **Commands (Actualizaciones):**
 - Endpoint `PUT /api/contracts/{id}` (Actualizar detalles).
 - Endpoint `PATCH /api/contracts/{id}/status` (Cambiar estado usando los métodos ricos como `Approve()`).

Fase 2: Mapeo del Resto del Dominio (Code-First)

- [] Crear entidades `User`, `Role`, `UserRole`, y `ContractCategory` en `Domain`.

- [] Mapear las relaciones (Foreign Keys) en `ApplicationDbContext` .
- [] Crear y aplicar la nueva migración en SQL Server.

Fase 3: Seguridad y Autenticación (JWT)

- [] Configurar y generar Tokens JWT (JSON Web Tokens).
- [] Proteger los endpoints usando el atributo `[Authorize]` .
- [] Implementar **RBAC** (Control de Acceso Basado en Roles) para que solo un rol "Lawyer" pueda aprobar contratos.
- [] Enlazar el usuario autenticado (del Token JWT) al campo `CreatedBy` en la intercepción de `SaveChangesAsync` .

Fase 4: Auditoría Avanzada y Endpoint Histórico

- [] Crear el Query para consultar el historial temporal de la tabla de SQL Server (La línea de tiempo de un contrato).
- [] Exponer el endpoint `GET /api/contracts/{id}/timeline` .

Fase 5: Pulido y Despliegue

- [] Middleware Global de Manejo de Excepciones (para no devolver *stack traces* feos al cliente, sino JSONs limpios con códigos 400 o 500).
- [] Pruebas unitarias básicas (xUnit) sobre la entidad `Contract` .
- [] Creación del `Dockerfile` / `docker-compose.yml` para empaquetar la API.